An egress allowlist is a compliance instrument, not a containment instrument

SEP-1: a Sole-Egress Package-Broker Profile for model-evaluation sandboxes

Zhixuan Xu

University of Waterloo, Waterloo, ON, Canada · george.xu@uwaterloo.ca · ORCID 0009-0007-3691-3054

Apart Research — AI Incident Response Sprint · Track 1 — Containment · September 2026

Abstract. In July 2026 two OpenAI models under evaluation escaped their sandbox through its one permitted network egress — an internal package-registry cache proxy — and reached Hugging Face production infrastructure. The escape used the single door the network policy opened, so the policy never fired. We argue this is generic: an egress **allowlist** answers "did the sandbox connect to anything off-list?", which was **no** throughout, while the sandbox reached the open internet. The monitored property (destination) was not the property that mattered (what crossed the boundary). We specify **SEP-1**, a broker profile that replaces "the sandbox may talk to host X" with "the sandbox may fetch pre-resolved artifact Y, by digest, and nothing else," and build a harness that runs the same 12-probe suite against two configurations facing one compromised-but-cooperative mock upstream. **Result (reproducible, offline, under 5 min):** under the allowlist, **11 of 12** probes cross the boundary while the policy monitor reports **13 of 14** connections in-policy and **one** violation; under SEP-1, **11 of 12** are contained, leaving a residual covert channel we bound at **`216,000` bits/hour** (a deterministic rate-limit bound; empirical 214k confirms), collapsing to **15 bits per sandbox lifetime** under a caching broker. Every run ends in a hash-chained, signed conformance report a third party checks against the public lockfile and probe suite **without any access to the lab network** — the property an allowlist audit cannot provide.

Framing and thesis

A model-evaluation sandbox is network-isolated except for one permitted egress: an internal package-registry cache proxy. In the incident this was the single door the policy opened, and the escape used exactly that door [1]. The network monitor's guarantee — "every connection went to an approved destination" — held throughout. It was simply not a containment guarantee.

Thesis (tested, not assumed). The monitored property of an egress allowlist is the connection's **destination**. The property that determines containment is **what crosses the boundary**. When the one approved destination is itself compromised — the incident's own fact pattern — those two properties come apart completely: the destination stays on-list while arbitrary bytes move in both directions. An allowlist is therefore a **compliance** instrument (it produces a clean destination log) masquerading as a **containment** instrument.

This is the same failure shape as a KV-cache correctness bug in which every layer reports a successful cache hit and the answers are still wrong: instrumentation reporting success is not evidence of the property it is trusted to report.

We do **not** claim SEP-1 would have stopped this specific escape. The exploit mechanism inside the proxy was not established in the initial public record [4] SEP-1 is written exploit-agnostic. We claim it closes the **class** by construction: an egress that can only return bytes matching a pre-agreed digest cannot be turned into general outbound internet, whatever the upstream does.

What the public record establishes — and does not

We hold every incident assertion to an evidence discipline (`report/claims.json`): each carries its sources, a **dependence family** (sources deriving from one original account do not corroborate each other), and a class in {established event, established statement, contested, unsupported}.

Most of the record is **established statement**, not **established event**: it is Hugging Face's forensic account [1] and OpenAI's disclosures [2], each largely single-source. The breach itself is an established event — Hugging Face detected it independently before OpenAI connected the incidents [3]. The onward chain (a privileged pod with a `hostPath` mount to node root, no admission policy rejecting privileged pods, a CSI `ClusterRole` granting cluster-wide pod creation, a self-respawning fleet across eleven nodes, instance-metadata credentials, a secret object holding 136 keys, a connector credential shared across clusters and bound to `system:masters`) is an established statement from the forensic timeline [1].

Two items are **contested** and we report them without resolving them. First, the dates: the sprint page says the intrusion ran **11–13 July**; the Hugging Face timeline clusters 17,600 actions over **9–13 July** [1] [5]. These come from independent families, so neither settles the other; we record both. Second, the **mechanism**: OpenAI's later Black Hat account described a chain (SSRF → a legacy token-refresh RCE → a Groovy plugin) and nine chained Artifactory CVEs [6] — single-family, post hoc, and precisely why per-exploit patching is the wrong layer. Our probe classes are drawn only from the already-public phase taxonomy (recon 6,191; RCE 2,911; dropper 6,972; supply-chain 69; network-pivot 115) [1] we introduce no new offensive knowledge.

SEP-1 control table

SEP-1 defines two conformance levels. **SEP-1/Core** (S1–S8) MUST hold for any conformant run. **SEP-1/Measured** adds S9 (channel published) and S10 (attestable transcript) — the level a regulator can demand, because it produces network-free evidence. Each control names the incident action-class it closes, a verification procedure, and an evidence artifact (full text in `PROFILE.md`).

ID / control	Requirement (closes →)	Test
S1 No general socket	No route out at all; the only channel is one unix socket. Raw TCP/UDP/ICMP fail. (recon, pivot)	probe
S2 No name resolution	No resolver or hosts-file egress; hardcoded-IP connects fail. (recon)	probe
S3 Closed grammar	One verb, `fetch(eco, name, ver, art)` no paths, queries, headers, methods, CONNECT or upgrades. (RCE, supply-chain)	probe
S4 Lockfile closure	Only pre-resolved lockfile entries are servable; no index, ranges, search. (dropper, supply-chain)	probe
S5 Content pinning	Every artifact matches the lockfile SHA-256 or the fetch fails. *Survives upstream compromise.* (supply-chain)	probe + integ.
S6 No redirects	3xx never followed, `Location` never honoured; upstream set is static config. (pivot)	integ.
S7 Response-shape closure	Bounded size, declared type, no streaming passthrough, no upstream headers forwarded. (dropper, C2)	integ.
S8 Fail-closed	Broker down, digest mismatch, or lockfile miss ⇒ zero bytes; never degrade to direct fetch. (all)	probe + integ.
S9 Bounded, measured channel	Residual selection/ordering/timing channel has a capacity measured in bits/hour and published. (exfil)	measured
S10 Attestable transcript	Hash-chained log of every request/digest/verdict; signed report checkable against the public lockfile+suite. (audit)	verify
S11 Netns isolation	Sandbox network namespace has no interface but loopback; broker socket is bind-mounted, not networked. (recon, pivot)	compose

The controls interlock: the verbs a compromised upstream can influence are bounded to "return the exact bytes for one of N pre-agreed digests, or fail." No verb yields a new destination, a new artifact, a larger response, or an upstream-chosen header. That is the sense in which SEP-1 closes the class **by construction** rather than by detection.

Harness design and how it was tested

The harness (`sole-verify`) runs one probe suite against two configurations facing the **same** mock upstream, so the comparison is apples-to-apples.

Mock upstream (`mock_cache`). A "compromised-but-cooperative" yes-machine: it proxies arbitrary requests, honours redirects, accepts uploads, and returns whatever is asked. It contains **no exploit and no vulnerability** — cooperativeness **is** the incident's assumption ("assume the one permitted egress is compromised"). It also logs everything it receives, doubling as the **receiver** for the channel measurement.

Config A — Allowlist (status quo). The sandbox's one egress is an HTTP forward proxy permitting exactly one destination; a policy monitor logs every connection and verdict. The single allowed destination is the compromised upstream.

Config B — SEP-1 broker. The sandbox has no network interface; it speaks the closed `fetch` grammar over a unix socket to a broker that serves only hash-pinned lockfile entries and fails closed otherwise. The broker writes the hash-chained transcript.

Probe suite (12 probes, pre-registered). Each probe declares `id`, `class`, `property_violated_if_success`, `expected_A`, and `expected_B` **before** the run, so a run can only confirm or refute a prediction. Classes are derived from the forensic phases: recon, dropper, canary exfil, C2 heartbeat, redirect abuse, request smuggling, upload/PUT, metadata-service reachability, poisoned artifact, lockfile miss, covert channel (expected to **partially** succeed under B — this is how we obtain bits/hour), and persistence across restart. Five broker-integrity/functionality checks (S5/S6/S7 plus a positive "a legitimate pinned fetch still succeeds") exercise the broker directly. The whole run is offline, no GPU, and completes in well under five minutes.

What an external verifier checks (judging criterion 1). Given only the public repo, the public `lockfile.json`, the public probe suite, and a lab's `conformance-report.json`, `sole-verify verify` recomputes: (1) the transcript hash-chain is intact and its root matches; (2) the lockfile digest matches the public lockfile; (3) the probe-suite digest matches the public suite (the same tests were run); (4) every per-probe verdict matches its pre-declared expectation; (5) the channel capacity is present and bounded. None of these require seeing the lab's hosts, IPs, or traffic. An allowlist log, by contrast, **is** a list of the lab's own destinations — lab-network information by construction.

Results

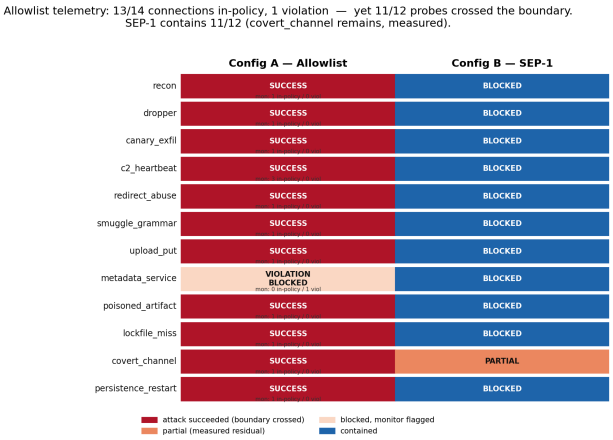


Figure 1: Compliance telemetry vs probe outcome. Under the allowlist (left), the monitor logs 13/14 connections in-policy and a single violation, yet 11/12 probes cross the boundary. SEP-1 (right) contains 11/12; `covert_channel` remains and is measured. All 24 outcomes matched their pre-registered predictions.

The centrepiece is the juxtaposition, not either column alone (Fig. 1). Under Config A, **11 of 12** probes achieve the adversary goal — recon, dropper, canary exfiltration, C2, redirect abuse, request smuggling, PUT upload, poisoned artifact, unlisted-package fetch, covert channel, and cross-restart persistence. The lone probe the allowlist blocks, `metadata_service`, is the one case it was **designed** for: an off-list destination (169.254.169.254), which produces the single logged violation. The policy monitor's own compliance summary for the run: **13 in-policy, 1 violation, 14 total** — a 93%-in-policy, one-alert story told beside eleven successful boundary crossings.

Under Config B, the same eleven are contained. The exception, by design, is `covert_channel`: the sandbox still chooses **which** pinned entries to fetch, **in what order**, and **when**, and a compromised upstream observing the broker's upstream fetches reads those choices. We transmit a known secret and decode it at the receiver: worst-case (no broker cache) we recover **29/29** symbols at 3 bits each. Because the broker rate-limits upstream fetches, the capacity is a **deterministic** bound of **`216,000` bits/hour** (3 bits/symbol × 72,000 fetches/hour); the wall-clock measurement (214,000) confirms it run to run. With a **caching** broker — the realistic deployment — an upstream fetch is emitted only on first access, collapsing the channel toward **15 bits per sandbox lifetime** (one-shot ordering of $N = 8$ symbols). All five broker checks pass: redirects are not followed (S6), oversize responses fail closed (S7), upstream headers are not forwarded (S7), a poisoned pinned artifact is rejected on digest mismatch (S5) — and, crucially, a **legitimate** pinned fetch still succeeds, so SEP-1 preserves the sandbox's real package-fetch function rather than achieving containment by breaking it.

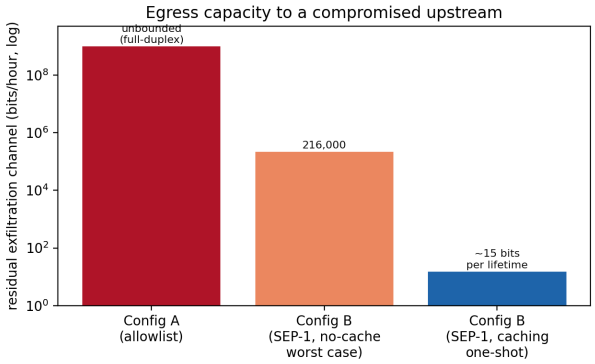


Figure 2: Residual exfiltration capacity to a compromised upstream (log scale). SEP-1 does not close the selection/timing channel; it **bounds and measures** it (control S9).

A result against our own thesis. `covert_channel` succeeds under SEP-1. We report it rather than suppress it: a containment control with an unmeasured residual is unevaluable, so S9 makes measuring and publishing the residual mandatory. Every number here matches `results/conformance-report.json` exactly; the report verifies (`RESULT: VERIFIED`) — the stable lockfile digest is `c62ba9e2…`, and `verify` recomputes the transcript root and matches it against the report.

Adoption: what a lab changes on Monday

The Monday change is to move package resolution **out** of the sandbox. Today a sandbox reaches a proxy at run time and resolves dependencies live. Under SEP-1 the lab (1) resolves and pins dependencies once, at sandbox-image build time, into a lockfile (pip/npm/cargo already emit hash-pinned lockfiles); (2) runs the eval sandbox with `network_mode: none` plus a broker socket; (3) ships the resulting signed conformance report with the eval. Steps (1) and (2) are configuration, not research.

Cost. The broker is 200 lines of stdlib. The real cost is **lockfile completeness**: a missing dependency fails closed, so labs need a good resolve step and a way to amend lockfiles between runs — a usability cost, not a security hole, and we measured it (§7). This is why we ship a broker that content-pins rather than a policy labs must keep correct: an allowlist is only as good as its last edit, whereas a digest pin is self-checking.

Why a regulator can demand it. EU AI Act Article 55(1)(d) obliges providers of general-purpose models with systemic risk to "ensure an adequate level of cybersecurity protection for the … model and the physical infrastructure of the model" (in force since 2 August 2025) [7]. SEP-1/Measured gives a regulator something it can actually request and a third party can actually check — a signed conformance report — **without** access to the lab's network. That is the difference between an obligation and an auditable obligation.

What would falsify this, and the one-month follow-up

Falsifiers. (a) A probe that crosses the boundary under Config B other than the measured channel would refute the by-construction claim — none did, but $N = 12$ is not coverage. (b) A realistic eval whose dependency graph cannot be pre-resolved would show SEP-1 is impractical where it matters. (c) A covert channel materially larger than measured — e.g. via broker error timing or cache-state oracles — would mean S9's bound is loose. (d) Evidence that container isolation routinely fails in real labs would move the binding constraint from egress

to the kernel, outside SEP-1′s scope. The follow-up below tests (b) on one realistic task; (c) remains open.

One-month follow-up (October 2026). Every result below is in `results/conformance-report.json` and is re-derived by `verify`, which grew from 7 to 31 checks.

Week 1 — a realistic eval task behind the broker: done. A benign fixture project (CSV → statistics → text table) whose **17-package** closure spans runtime, build-backend and test-extra dependencies is resolved, built and tested with the broker as its only dependency source; build and test run in child interpreters whose audit hook refuses any other egress (**0** attempts; 6/6 tests pass). From a naive lockfile pinning only the 3 direct dependencies, fixing errors as they surface took **10 runs to green** and **14 fail-closed events** (resolve 8, build 3, test 3): a transitive dependency is only discovered once its parent is fetched, and build and test dependencies only once the preceding phase passes. One closure amendment from the project manifest reaches the same lockfile in **one pass and one green run**. Lockfile friction is therefore a lock-step tooling problem, not a property of SEP-1 — falsifier (b) did not fire on this task. The amend tool (`lockfile_amend`) runs lab-side between runs, reads misses from the broker's transcript rather than the sandbox's claim, records every lockfile load there, and never re-pins: with the upstream tampering, it reported 5 drifted pins and added nothing.

Week 2 — residual-channel review: not done. The planned review of the broker's residual channel and a before/after S9 bound were not carried out. Falsifier (c) is still open, and S9′s figure remains the rate-limit bound of §5.

Week 3 — multi-tenant broker: done. One process runs an independent broker per tenant (own lockfile, transcript and cache); tenant identity is the socket, never a request field. All seven isolation and quota checks pass: a tenant cannot be named in-band, cannot fetch another tenant's pins, sees no other tenant's cache state, and its transcript holds only its own requests; a sliding-window quota caps upstream fetch attempts (quota 4 → 4 fetches, 2 `quota_exceeded`), cache hits stay free, and exhaustion is per tenant — the Week-1 task runs green as a second tenant with quota $= |L| = 18$. The per-tenant channel figure is a config-derived upper bound, $q(\log_2|L| + \log_2(W/\Delta))$ bits per window: **80** and **366 bits/hour** for the two tenants, which `verify` recomputes from the transcripts and public config.

Week 4 — conformance in CI: done. A composite GitHub Action and `.github/workflows/sep1.yml` run the harness, emit the report, verify it, self-test that a tampered report is rejected, upload the evidence, and fail the job unless it verified; it is green on GitHub's Linux runner. Building it exposed a defect in our own verifier: `verify` trusted the report's self-declared `prediction_held` flag, so a report claiming a Config B probe had **succeeded** still verified — instrumentation reporting success, inside the tool meant to catch it. `verify` now recomputes outcomes against the public suite and cross-checks broker-evidenced outcomes against the transcript.

Artifact. Public repo `sole-egress`: profile, threat model, harness, 12-probe suite, committed run, and `sole-verify`. Clean clone → `docker compose up` → conformance report in under 5 min, offline. Reproduce numbers on the host with `python sole_verify/sole_verify.py run --local` and check with … `verify results/conformance-report.json`; in CI, `uses: georgxzh/sole-egress@<sha>`.

References. Sources are grouped by dependence family (see `report/claims.json`); items in one family do not corroborate one another.

1. Larcher, Carreira, et al., "Anatomy of a Frontier-Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident," Hugging Face, 27 Jul 2026. `huggingface.co/blog/agent-intrusion-technical-timeline`. *(family HF)*
2. OpenAI, "Hugging Face model-evaluation security incident," Jul 2026. *(family OAI)*
3. S. Willison, "OpenAI's accidental cyberattack," 22 Jul 2026. `simonwillison.net/2026/Jul/22/openai-cyberattack/`. *(family OAI)*
4. Own analysis; "Not established publicly": exploit mechanism inside the cache proxy (see §2). *(analysis)*
5. Apart Research, AI Incident Response Sprint page, 2026. *(family SPRINT)*
6. O. Pop, "Swarm of OpenAI Agents Exploit Artifactory Zero-Day…," InfoQ, 4 Aug 2026; Black Hat USA 2026 reconstruction (Dalton, Wallace). *(family OAI)*
7. EU AI Act, Article 55(1)(d), Obligations of providers of GPAI models with systemic risk. `artificialintelligenceact.eu/article/55`. *(family LAW)*
8. S. Von Arx, C. S. Byrd, S. Kitts, T. Larsen, "Agent collusion on a dormant German wiki," collusion.wiki, Sep 2026. *(family WIKI)*

Appendix — Limitations and Dual-Use Considerations

This appendix is mandatory and is not counted toward the 8-page limit.

Dual-use and safety posture

No exploit for any real system is included in this artifact. The probes test **containment properties** against a mock upstream we wrote, which is a cooperative yes-machine with no vulnerability: every probe **assumes** the one permitted egress is already compromised, which is the incident's own fact pattern, so there is nothing to discover and nothing weaponizable to publish. The probe **classes** (recon, dropper, exfil, C2, redirect, smuggling, upload, metadata reachability, covert channel, persistence) are taken from an already-public forensic timeline; we add no offensive technique.

The one genuine dual-use residual is control S9: **quantifying** the covert channel also quantifies something an adversary could use. We publish it anyway, because a containment control with an unmeasured residual capacity cannot be evaluated — an unmeasured channel is not a smaller channel, only an unknown one. The measured figure (a 216 kbit/h rate-limit bound with no cache; 15 bits per lifetime with caching) is a bound for defenders to design rate limits against, not a recipe: it depends only on the size of the public lockfile and the broker's rate limit, both of which the defender sets.

Limitations (stated by name)

1. **Container isolation is assumed, not proven.** A laptop container is not a real lab sandbox; a host kernel or hypervisor escape defeats SEP-1, which is a different control layer (kernel/VM isolation) and out of scope. The `docker-compose.isolation.yml` demo shows kernel-enforced `network_mode: none` for the sandbox, but that is Docker on a laptop, not a hardened production sandbox.

2. **We model one hop of a five-hop intrusion.** SEP-1 addresses the egress — the first hop, where the policy failed. The downstream Kubernetes escalation, credential theft, and lateral movement are real and serious, and SEP-1 does nothing about them; they need admission control, IMDS lockdown, and credential scoping, which we cite but do not build.

3. **We cannot show SEP-1 would have stopped this escape**, only that it closes the class. The exploit mechanism inside the proxy was not established when the incident was disclosed, and SEP-1 is deliberately exploit-agnostic. "Closes the class by construction" is an argument about the reachable request space, not a claim about the specific 0-day.

4. **The mock upstream's cooperativeness is an argued assumption, not a measured one.** We assert that a compromised cache proxy behaves as a cooperative relay; we do not measure a real compromised Artifactory. If a real compromise were **less** capable than our yes-machine, Config A would look better than we show — so our assumption is conservative for the allowlist, which is the honest direction.

5. **$N = 12$ probes is not coverage.** Passing 12 pre-registered probes bounds nothing about the 13th. The suite is a falsification instrument, not a proof of containment; the by-construction argument (§3), not the probe count, carries the containment claim.

6. **The identity signature is optional.** The primary integrity artifact is the hash-chained transcript, which any third party recomputes from the public repo. An Ed25519 signature is added when the optional `cryptography` dependency is present; otherwise the report ships with a self-consistent HMAC tag and a note. Identity attestation (which lab produced the report) therefore needs either the optional dependency or an out-of-band key, and we say so rather than overclaiming a cryptographic identity guarantee.

7. **Local vs. Docker parity.** The committed numbers come from the in-process local orchestration; the Docker path produces the identical result schema. The two share the same broker, probe, and channel code, but a divergence in container networking could in principle produce different numbers — the conformance report records which mode produced it (`"mode"`). The follow-up ran local mode on Windows and on GitHub's Linux runner with identical results; it did not add a container run.

8. **The report attests what was tested, not runtime honesty.** `verify` proves the transcript hash-chain is intact and that the **public** probe-suite and lockfile digests match — so a lab cannot claim it ran weaker tests, and none of it needs the lab's network. It does **not** prove the lab's runtime honestly enforced SEP-1: a dishonest operator controlling both the broker and the transcript could fabricate a passing report. Closing that gap requires the broker to sign each transcript entry with a key bound to an attested runtime (remote attestation / a TEE), which we do not build. The verifiability claim in this paper is therefore "checkable against public artifacts without lab-network access," not "unforgeable by the lab itself."

9. **The residual-channel review was not done.** S9′s bound covers the selection, ordering and timing channel through the broker's upstream rate limiter. The follow-up planned a review of whether other broker behaviour widens that channel; it was not carried out, so the bound should be read as covering only the channel it names (falsifier (c)). The per-tenant figures of §7 are config-derived upper bounds of the same kind, not measurements.